

KINGDOM OF MOROCCO
Ministry of Higher Education
Engineering Sciences - Computer Engineering / Software Engineering

FINAL YEAR ENGINEERING PROJECT (PFE)

MyHeath

Design and Implementation of an Intelligent Telemedicine and Women's Health Tracking Platform

MERN Stack | Docker | AES-256 Encryption | Predictive Analytics | Claude AI | Vercel Cloud

Author: Salaheddine Admo
Email: salaheddine.admo@gmail.com
Academic year: 2025 - 2026

Submitted in partial fulfillment of the requirements for the Engineering Degree

Dedication

To my family, whose unwavering support made this journey possible.

To every woman who deserves accessible, respectful and secure digital healthcare.

To my professors and mentors who shaped my engineering mindset.

Acknowledgments

I would like to express my sincere gratitude to my academic supervisor for continuous guidance, constructive feedback and scientific rigor throughout this Final Year Project. I also thank the faculty members of the Computer Engineering / Software Engineering department for the solid theoretical and practical foundations provided during my studies.

Special thanks to classmates and peers who reviewed early prototypes of MyHeath and shared valuable usability feedback. Finally, I acknowledge the open-source communities behind Node.js, React, MongoDB, Docker and Anthropic Claude, whose tools made this engineering work achievable.

Abstract (English)

MyHeath is an intelligent FemTech telemedicine platform designed to improve women's remote healthcare through predictive menstrual-cycle analysis, encrypted medical records, secure patient-doctor messaging and an AI health companion powered by Anthropic Claude. The system follows a modular MERN architecture (MongoDB, Express.js, React, Node.js), with AES-256-CBC encryption at rest for sensitive clinical fields, JWT-based authentication and role-based access control (Patient, Doctor, Admin).

A deterministic predictive engine computes average cycle length, next-period forecasts, ovulation windows and anomaly alerts related to irregular cycles and recurring severe pain (early signals associated with PCOS and endometriosis). MyHeath AI contextualizes answers using encrypted patient history to provide insight narration, natural-language symptom parsing, doctor briefs and personalized wellness plans. The frontend is deployed on Vercel; the API is deployed as Vercel serverless functions connected to MongoDB Atlas.

Keywords: FemTech, telemedicine, MERN, AES-256, predictive analytics, Claude AI, Vercel, PFE

Resume (Francais)

MyHeath est une plateforme de telemedecine FemTech visant a ameliorer le suivi a distance de la sante feminine via une analyse predictive du cycle, un dossier medical chiffre, une messagerie securisee et un assistant IA (Claude). L'architecture MERN modulaire integre chiffrement AES-256-CBC, JWT/RBAC, moteur d'alertes SOPK/endometriose et deploiement cloud Vercel + MongoDB Atlas.

Mots-cles: FemTech, telemedecine, MERN, AES-256, IA Claude, Vercel, PFE

Table of Contents

Dedication

Acknowledgments

Abstract / Resume

List of Figures and Tables

General Introduction

Chapter 1 - Preliminary Study and Requirements

1.1 Context and market analysis

1.2 Related work and comparative study

1.3 Problem statement and objectives

1.4 Functional requirements

1.5 Non-functional requirements

1.6 Actors and global use cases

Chapter 2 - Design and Modeling

2.1 Global system architecture

2.2 Technology choices justification

2.3 UML modeling

2.4 MongoDB data schema

2.5 Security design (AES-256, JWT, RBAC)

2.6 Predictive engine design

2.7 AI module design

Chapter 3 - Implementation

3.1 Development environment

3.2 Backend modules

3.3 Frontend modules

3.4 Encryption utilities

3.5 Predictive analyzer

3.6 MyHeath AI integration

3.7 Telemedicine messaging

Chapter 4 - Testing and Deployment

4.1 Test strategy

4.2 Functional validation scenarios

4.3 Security verification

4.4 Cloud deployment (Vercel + Atlas)

4.5 CI/CD and Docker

General Conclusion and Perspectives

Bibliography

Annexes

List of Figures

- Figure 2.1 - Three-tier MERN architecture of MyHeath
- Figure 2.2 - High-level deployment topology (Vercel + Atlas)
- Figure 2.3 - Use-case diagram (Patient / Doctor / Admin)
- Figure 2.4 - Sequence: login and JWT issuance
- Figure 2.5 - Sequence: symptom log and insights computation
- Figure 2.6 - Class model (users, records, logs, messages)
- Figure 2.7 - AES-256-CBC encryption pipeline
- Figure 2.8 - Predictive cycle analysis flowchart
- Figure 2.9 - MyHeath AI contextual prompt pipeline
- Figure 3.1 - Backend folder structure
- Figure 3.2 - Frontend route map
- Figure 3.3 - Dashboard cycle chart (Recharts)
- Figure 3.4 - MyHeath AI companion interface
- Figure 4.1 - Deployment architecture on Vercel

List of Tables

- Table 1.1 - Comparative analysis of FemTech / telemedicine solutions
- Table 1.2 - Functional requirements catalog
- Table 1.3 - Non-functional requirements
- Table 2.1 - Technology stack summary
- Table 2.2 - MongoDB collections overview
- Table 2.3 - RBAC matrix
- Table 2.4 - Anomaly detection rules
- Table 3.1 - REST API endpoints
- Table 3.2 - AI endpoints
- Table 4.1 - Test scenarios and results
- Table 4.2 - Environment variables for production

General Introduction

Project context

Women's health remains structurally under-addressed in many digital health ecosystems. Despite the rapid growth of FemTech worldwide, patients in medically underserved regions still face barriers: limited specialist availability, fragmented paper records, stigma around gynecological symptoms, and weak continuity between self-tracking and clinical care. Digital platforms can reduce these gaps when they combine trustworthy clinical support, privacy-preserving storage and actionable insights rather than generic wellness tips.

Morocco and the broader Maghreb region illustrate this tension. Mobile connectivity is widespread, yet specialized gynecological follow-up is unevenly distributed between urban and rural areas. A secure telemedicine layer, coupled with intelligent cycle tracking, can help patients prepare consultations, detect concerning patterns earlier and communicate safely with clinicians.

MyHeath was conceived as an engineering answer to this challenge: a production-oriented web platform that unifies predictive tracking, encrypted dossiers, telemedicine messaging and generative AI assistance within a clean modular architecture suitable for a Final Year Engineering Project (PFE).

Problem statement

How can we design and implement a women's telemedicine platform that (1) protects sensitive health data with strong cryptography and access control, (2) provides clinically meaningful predictive insights and anomaly alerts, (3) integrates an AI assistant without replacing human clinicians, and (4) is deployable on modern cloud infrastructure with a maintainable MERN codebase?

General and specific objectives

General objective: deliver a secure, modular FemTech telemedicine MVP of engineering quality.

- Specify functional and non-functional requirements for patients and doctors.
- Design a 3-tier MERN architecture with clear module boundaries.
- Implement AES-256-CBC encryption at rest for medical records and messages.
- Implement a rule-based predictive cycle engine with anomaly detection.
- Integrate Claude-based MyHeath AI for explanation, journaling and consultation prep.
- Deploy frontend and API on Vercel with MongoDB Atlas persistence.
- Document the system academically for PFE evaluation.

Methodology

The project followed an iterative engineering methodology inspired by agile delivery: requirements elicitation, architecture design, incremental implementation (auth -> health tracking -> encryption -> AI -> messaging -> deployment), continuous validation and technical documentation. Cursor AI was used as a development accelerator under human architectural supervision, which itself constitutes a modern software engineering practice relevant to industry workflows.

Report structure

Chapter 1 presents the preliminary study, market comparison and requirements. Chapter 2 details architecture,

UML views, database design, security and algorithm design. Chapter 3 describes implementation of backend, frontend, crypto, analyzer and AI. Chapter 4 covers testing and cloud deployment. The conclusion summarizes contributions and outlines future work.

Chapter 1 - Preliminary Study and Requirements Specification

1.1 Socio-technical context

FemTech encompasses digital products targeting fertility, menstrual health, pregnancy, menopause and related clinical pathways. Market analyses over the last decade show accelerating investment, but also recurring criticism: weak clinical validation, opaque data practices, and limited integration with licensed clinicians. For an engineering thesis, the opportunity is to demonstrate that privacy-by-design and medical caution can coexist with an engaging product experience.

Telemedicine adoption increased after COVID-19 and remains relevant for follow-up visits, triage and remote counseling. In women's health, asynchronous secure messaging is often more practical than video for symptom clarification and document sharing. MyHeath therefore prioritizes encrypted text consultation, while remaining extensible toward video (WebRTC) in future iterations.

1.2 Existing solutions and comparative study

Consumer apps such as Clue and Flo popularized cycle logging and predictions. Medical chatbots such as Ada focus on symptom assessment. Hospital portals provide records but rarely cycle intelligence. National e-health initiatives improve administration more than FemTech-specific journeys. MyHeath positions itself at the intersection of tracking, telemedicine, encryption and generative AI.

Solution	Tracking	Telemed.	Encryption focus	GenAI
Clue / Flo	Strong	Limited	Vendor-dependent	Partial
Ada	Weak	Triage chat	Standard	Yes
Hospital portal	Weak	Varies	Institutional	Rare
MyHeath (ours)	Strong	Yes	AES-256 at rest	Yes

Table 1.1 - Comparative analysis of FemTech / telemedicine solutions.

1.3 Stakeholders

- Patient: logs symptoms/periods, views insights, chats with doctor, uses AI assistant.
- Doctor: reviews assigned patients, receives messages, supports clinical follow-up.
- Admin (extensible): manages users and platform configuration.
- System: computes predictions, encrypts data, serves AI context securely.

1.4 Functional requirements

The functional catalog was derived from user stories and PFE engineering constraints.

ID	Requirement	Priority
RF01	Register/login with roles	Must
RF02	Assign doctor to patient	Must
RF03	Log symptoms and period events	Must
RF04	Compute cycle insights/alerts	Must
RF05	Encrypted health record CRUD	Must

RF06	Secure patient-doctor messaging	Must
RF07	AI chat with patient context	Should
RF08	NL symptom parsing	Should
RF09	Doctor brief generation	Should
RF10	Daily wellness plan	Could

Table 1.2 - Functional requirements catalog.

1.5 Non-functional requirements

ID	Category	Requirement
RNF01	Security	AES-256-CBC at rest for sensitiv
RNF02	Security	JWT + RBAC on protected routes
RNF03	Privacy	Minimize plaintext clinical stor
RNF04	Performance	API responses suitable for SPA U
RNF05	Scalability	Stateless API + managed MongoDB
RNF06	Portability	Docker Compose for local parity
RNF07	Usability	Responsive English UI
RNF08	Reliability	DB retry logic and healthchecks
RNF09	Maintainability	Modular controllers/routes/utils
RNF10	Deployability	Vercel frontend + serverless API

Table 1.3 - Non-functional requirements.

1.6 Regulatory and ethical considerations

Although MyHeath is an academic prototype, design choices intentionally mirror principles found in Morocco's Law 09-08 on personal data protection and international healthcare privacy practices (e.g., HIPAA encryption-at-rest guidance). The AI assistant always disclaims that it is not a physician and must not issue definitive diagnoses. High-severity anomaly alerts encourage human consultation rather than autonomous clinical decisions.

Ethically, women's health data is highly sensitive. Encryption, least-privilege access, and transparent AI limitations are therefore first-class requirements, not optional extras.

1.7 Scope and limitations

- In scope: web MVP, predictive rules, encryption, AI companion, messaging, cloud deploy.
- Out of scope (v1): video consultation, mobile native apps, certified HDS hosting, ML training.
- Clinical limitation: rule-based alerts are screening signals, not diagnostic tools.

1.8 Chapter synthesis

Chapter 1 established the need for a privacy-aware FemTech telemedicine platform and formalized requirements that drive architecture and implementation choices in subsequent chapters.

Chapter 2 - Design and Modeling

2.1 Global architecture

MyHeath adopts a classic three-tier architecture aligned with the MERN stack. The presentation tier is a React Single Page Application. The application tier is an Express API exposing REST endpoints and optional Socket.io locally. The data tier is MongoDB Atlas via Mongoose ODM.

Figure 2.1 — MyHeath 3-tier MERN Architecture

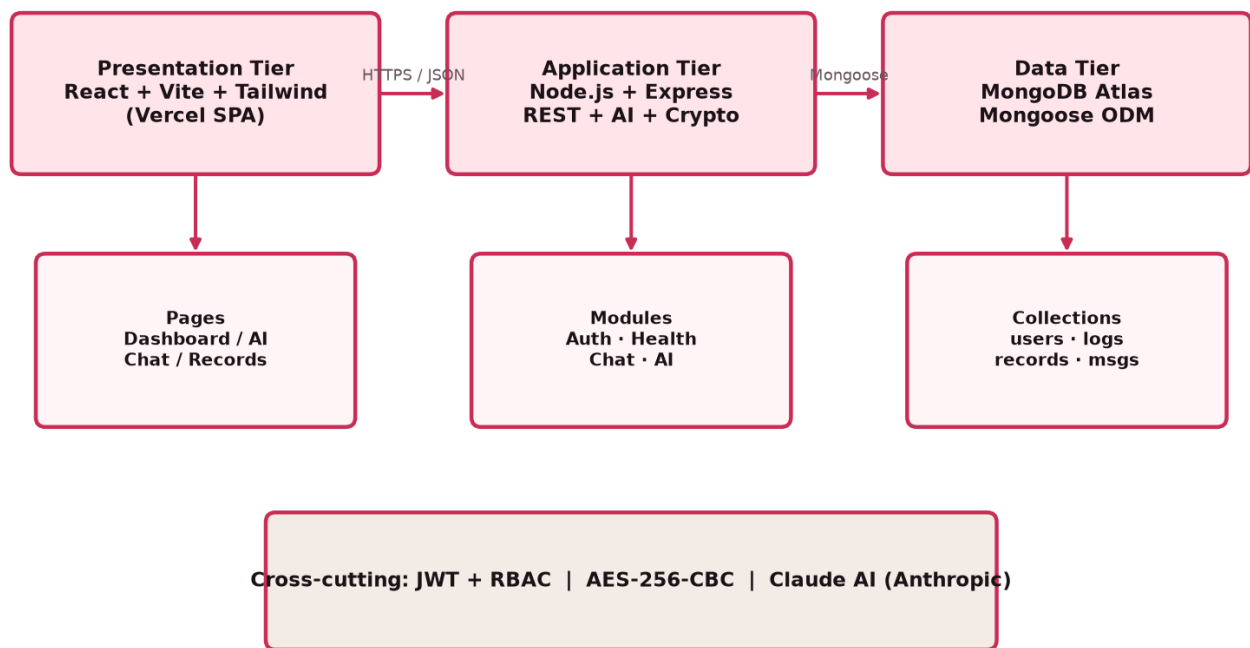


Figure 2.1 - Three-tier MERN architecture of MyHeath

On Vercel, the API runs as serverless Node functions. Because long-lived WebSocket connections are unreliable in this model, production messaging uses REST send + polling, while local Docker retains Socket.io for realtime demos.

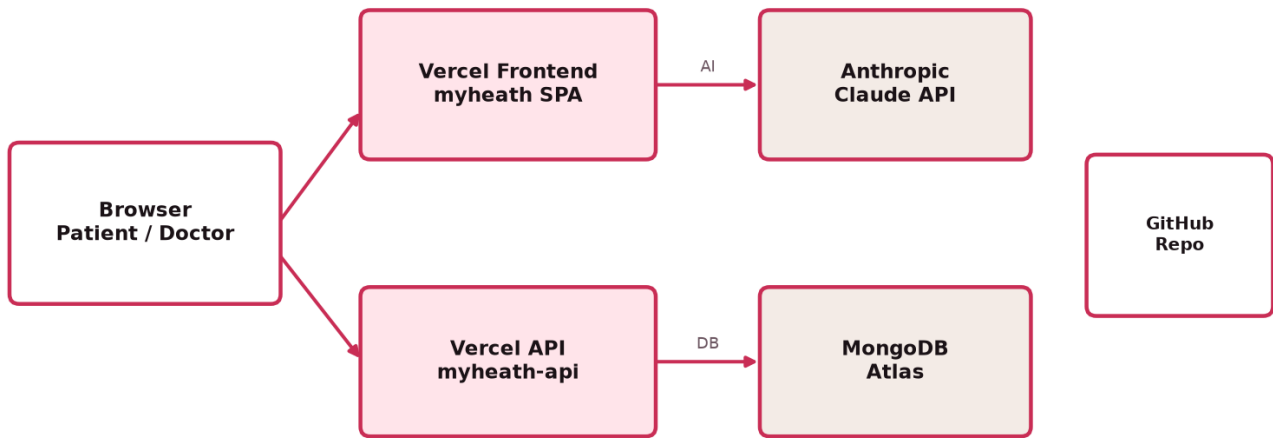
Figure 2.2 — Deployment Topology

Figure 2.2 - High-level deployment topology (Vercel + Atlas)

2.2 Technology choices

Layer	Technology	Rationale
Frontend	React 18 + Vite	Fast DX, component model
Styling	Tailwind CSS	Rapid consistent UI
Charts	Recharts	Cycle visualization
Backend	Node + Express	JS fullstack cohesion
DB	MongoDB Atlas	Flexible health event schemas
Auth	JWT + bcrypt	Stateless secure sessions
Crypto	Node crypto AES	Native, auditable
AI	Anthropic Claude	Strong medical-safe prompting
Deploy	Vercel	SPA + serverless Node
Local ops	Docker Compose	Reproducible environments

Table 2.1 - Technology stack summary.

MongoDB was preferred over a rigid relational schema because symptom journals evolve quickly (new symptom tags, optional temperature, free-text notes) and benefit from document flexibility. Indexes on (patient, date) preserve query performance.

2.3 UML - Use cases

Figure 2.3 — Use Case Diagram

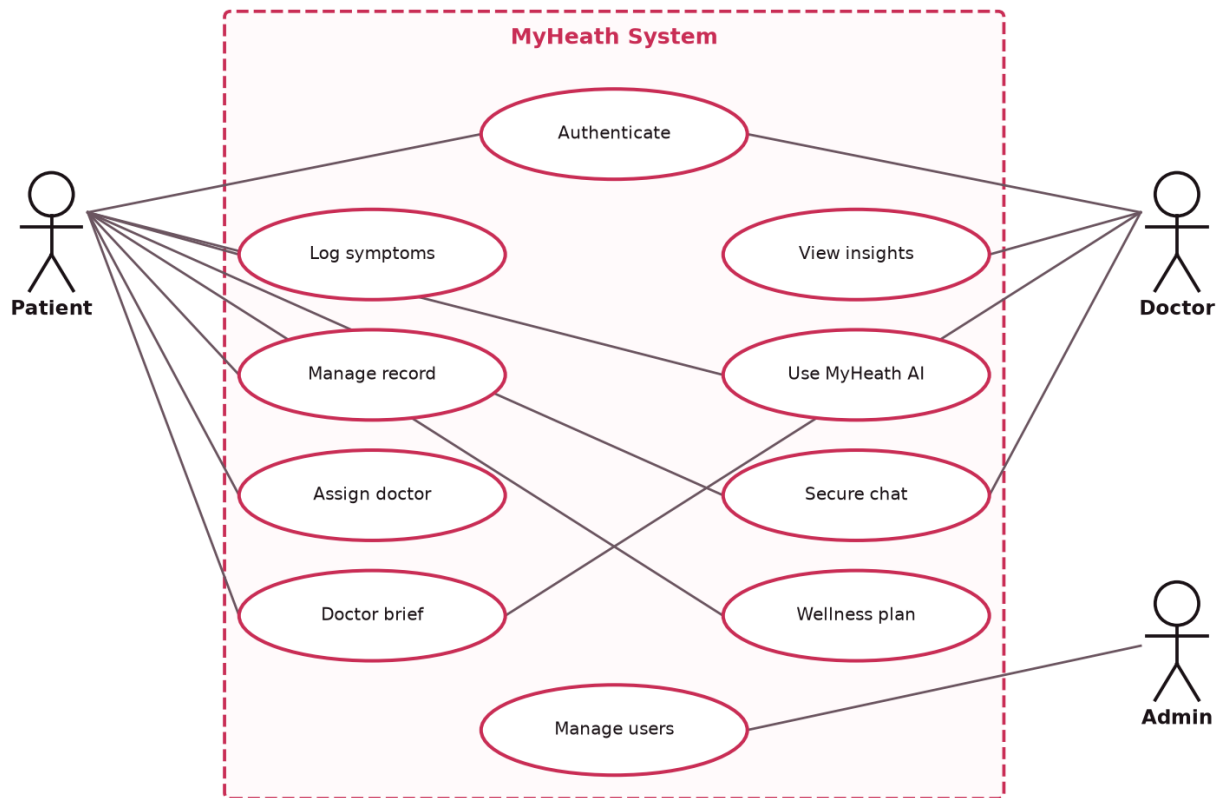


Figure 2.3 - Use-case diagram (Patient / Doctor / Admin)

Primary patient use cases include authentication, symptom logging, insight consultation, health-record updates, doctor assignment, messaging and AI assistance. Doctor use cases center on reading assigned conversations and supporting follow-up. Admin remains an extension point for user governance.

2.4 UML - Sequence diagrams

Figure 2.4 — Sequence: Login & JWT

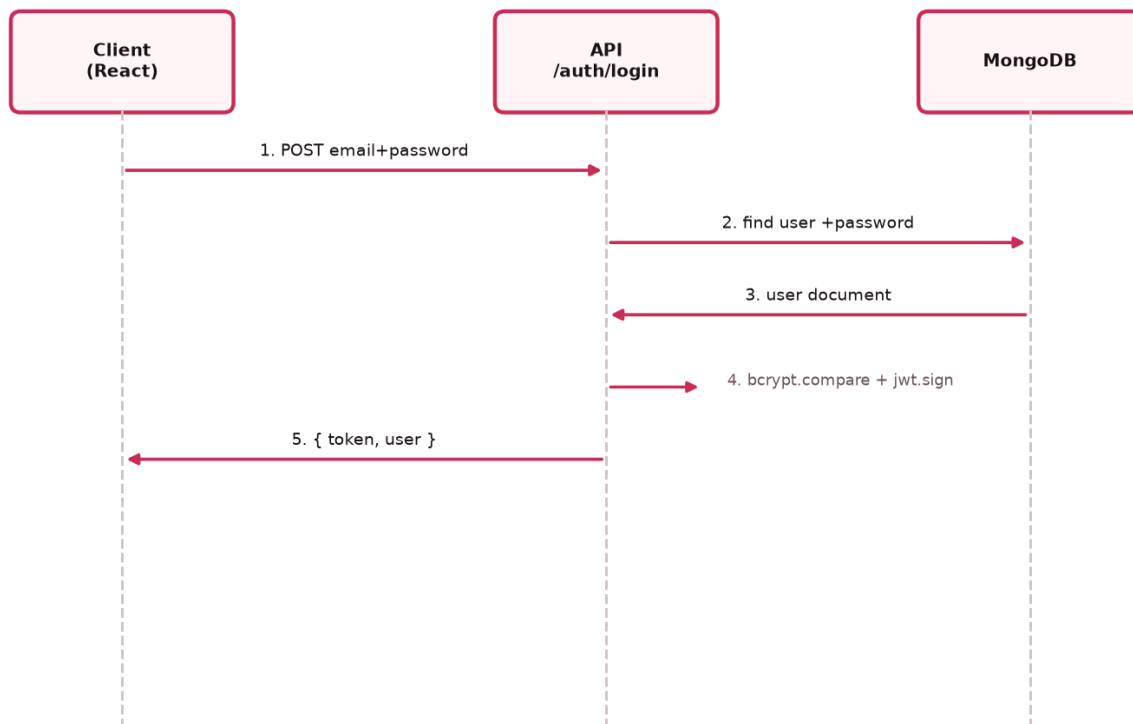


Figure 2.4 - Sequence: login and JWT issuance

During login, the API loads the user with password hash, verifies credentials using bcrypt, signs a JWT containing the user id, and returns a safe user DTO without the password field.

Figure 2.5 — Sequence: Insights Computation

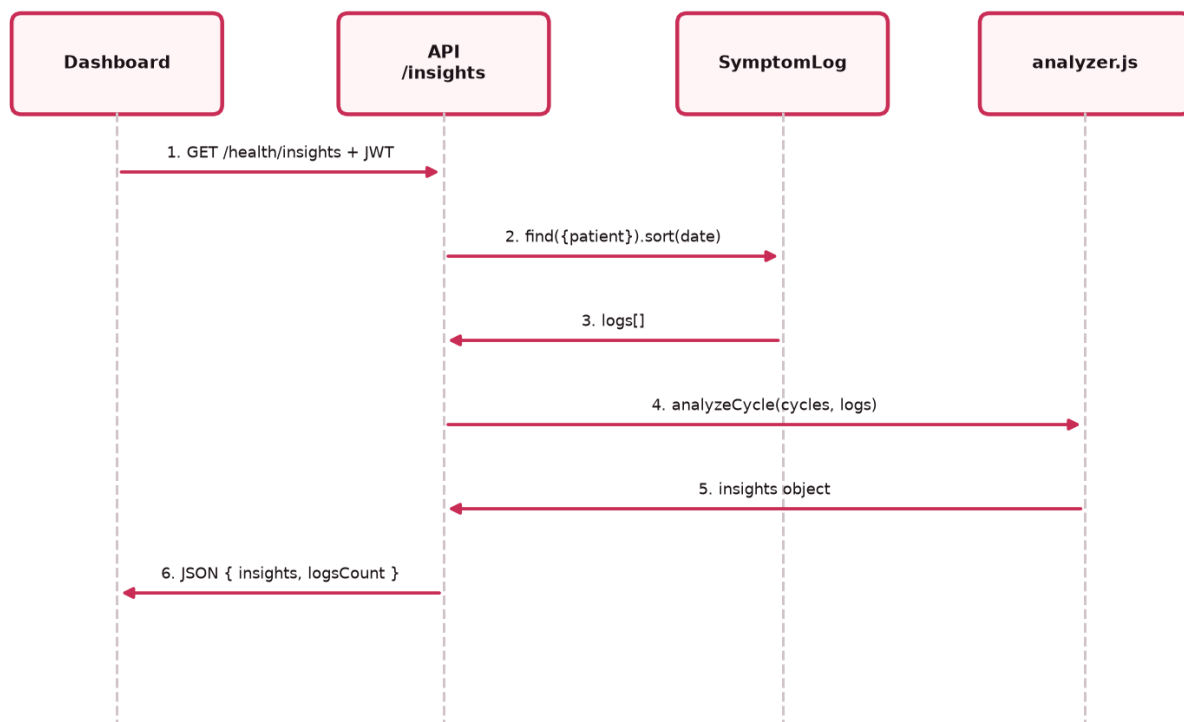


Figure 2.5 - Sequence: symptom log and insights computation

Insights computation is synchronous and deterministic: period_start events define cycles; symptom logs feed anomaly heuristics; the response includes chart series for the dashboard.

2.5 UML - Domain classes

Figure 2.6 — Domain Class Diagram

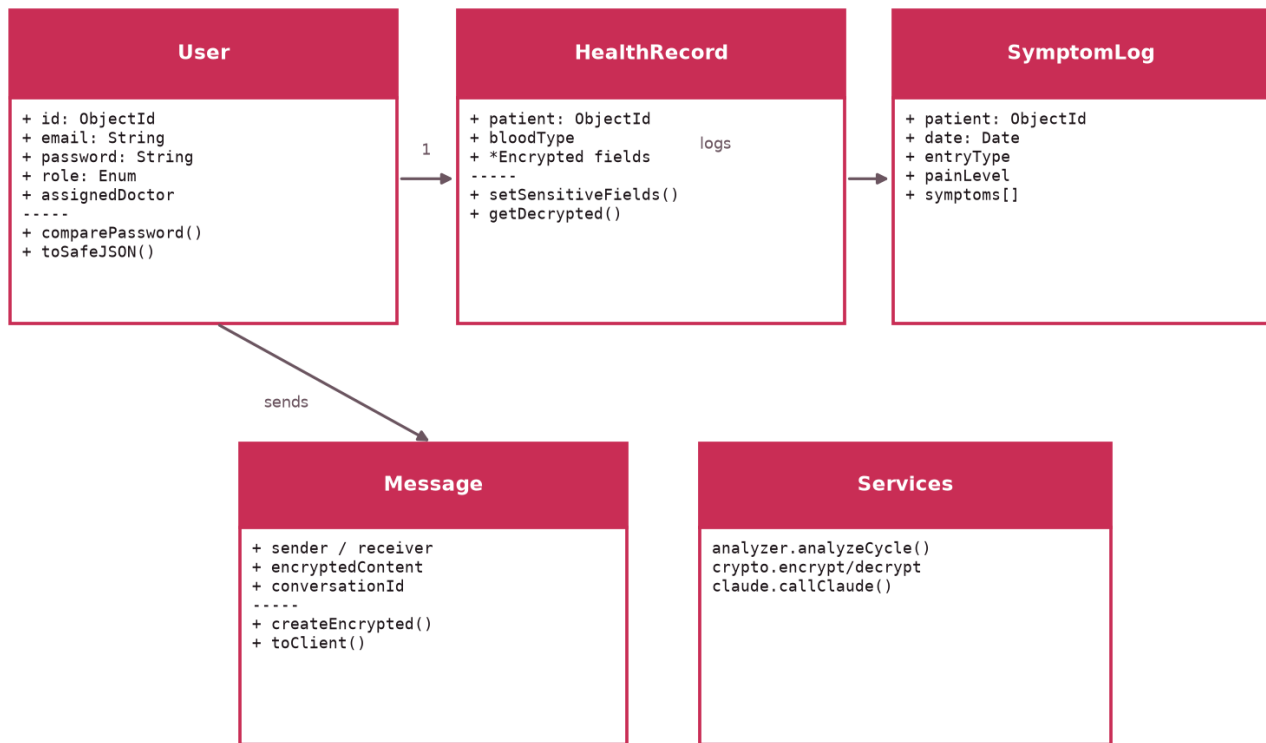


Figure 2.6 - Class model (users, records, logs, messages)

User holds identity and role. HealthRecord stores encrypted clinical fields. SymptomLog captures longitudinal observations. Message stores encrypted conversation content and a deterministic conversationId built from sorted participant ids.

2.6 Database design (MongoDB)

Collection	Key fields	Notes
users	email, password, role, assignedD	bcrypt hashed password
healthrecords	allergiesEncrypted, notesEncrypt	AES payload
symptomlogs	entryType, painLevel, symptoms[]	indexed by patient/date
messages	encryptedContent, conversationId	AES payload

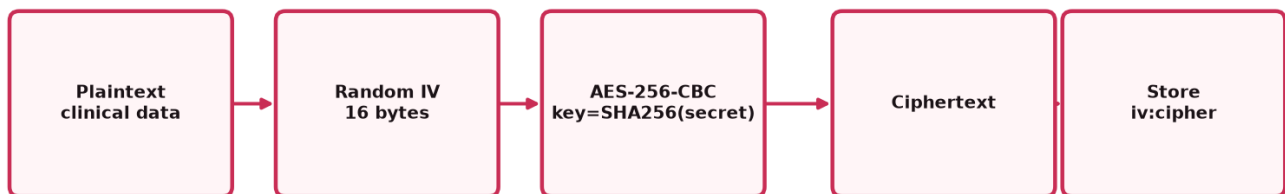
Table 2.2 - MongoDB collections overview.

NoSQL does not eliminate modeling discipline. We still enforce schema validation with Mongoose, unique email constraints, enums for roles/entry types, and explicit references between patients and doctors.

2.7 Security design

Security is layered: transport HTTPS in production, authentication via JWT bearer tokens, authorization via middleware authorize(...roles), password hashing with bcrypt cost 12, and field-level encryption for clinical content.

Figure 2.7 — AES-256-CBC Encryption Pipeline



Decrypt only on server after JWT + RBAC authorization

Figure 2.7 - AES-256-CBC encryption pipeline

The encryption utility derives a 256-bit key from AES_SECRET_KEY using SHA-256, generates a random 16-byte IV per encryption, and stores iv and ciphertext together. Decryption occurs only server-side after authentication. This protects database dumps and unauthorized direct collection reads.

Action	Patient	Doctor	Admin
Own insights	Yes	Assigned only	Yes
Update own record	Yes	No	Yes
Message assigned peer	Yes	Yes	Yes
AI tools	Yes	Limited	Yes

Table 2.3 - RBAC matrix (simplified).

2.8 Predictive engine design

The analyzer is intentionally rule-based for transparency and auditability in a PFE context. Machine learning can be added later once labeled clinical datasets are available.

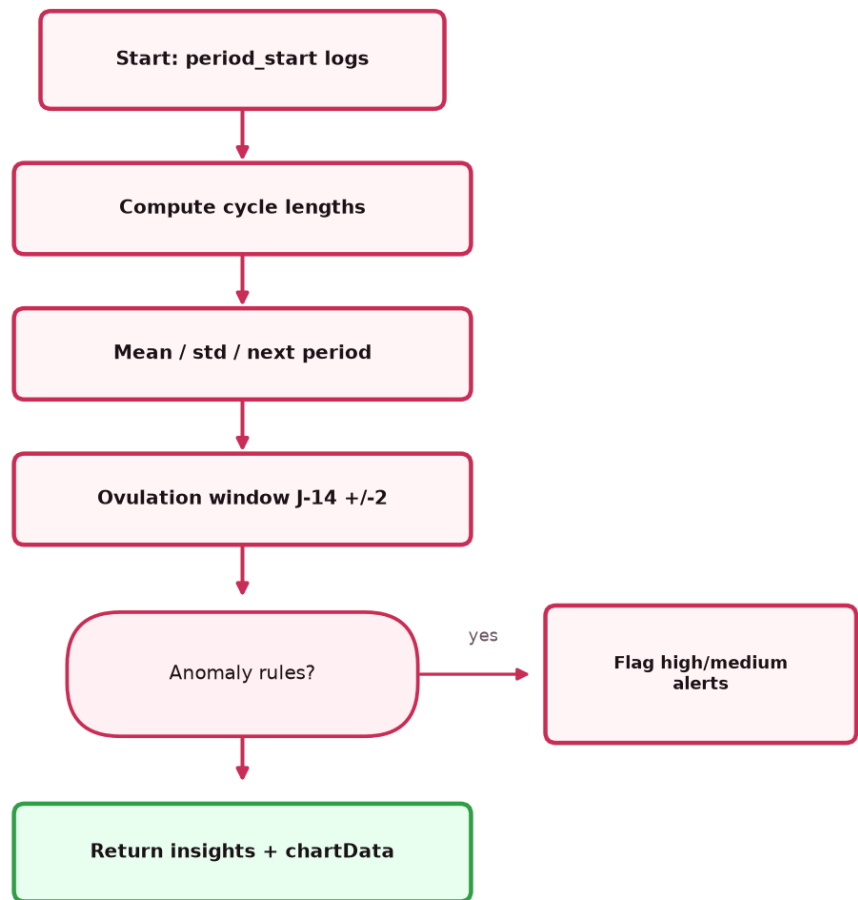
Figure 2.8 — Predictive Cycle Analysis Flowchart

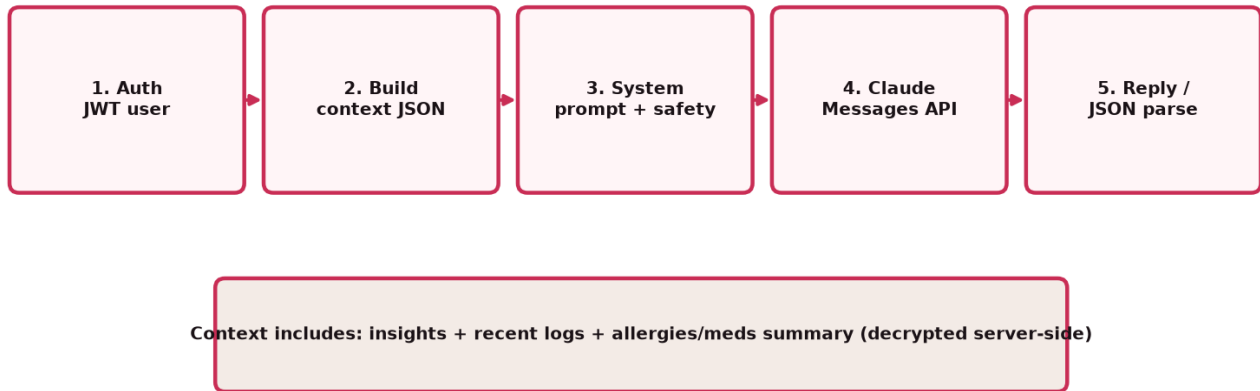
Figure 2.8 - Predictive cycle analysis flowchart

Rule	Condition	Severity
IRREGULAR_CYCLES	stdDev >= 7 days	High
ABNORMAL_LENGTH	mean <21 or >35	Med/High
SEVERE_PAIN	>=3 logs with pain>=7	High
PCOS_SIGNALS	symptom cluster + irregularity	Medium

Table 2.4 - Anomaly detection rules.

Ovulation estimation uses the classical luteal-phase approximation (peak near day -14 before next menses, with a +/-2 day fertile window). Patients are informed that this is statistical guidance, not a fertility medical device certification.

2.9 AI module design

Figure 2.9 — MyHeath AI Contextual Prompt Pipeline*Figure 2.9 - MyHeath AI contextual prompt pipeline*

The AI controller builds a confidential patient context JSON (insights, recent logs, non-sensitive record summary), injects a strict system prompt (no definitive diagnosis, escalate emergencies, encourage clinician consultation), and calls Anthropic Messages API. Structured features (parse, wellness, ask-doctor) require JSON-only answers parsed safely.

2.10 Chapter synthesis

Chapter 2 translated requirements into an implementable architecture with explicit security, data and algorithmic designs that guide the realization chapter.

Chapter 3 - Realization and Implementation

3.1 Development environment

- IDE: Cursor / VS Code
- Runtime: Node.js 20+/22
- Package managers: npm
- Database: MongoDB Atlas
- Local orchestration: Docker Compose (optional)
- Version control: Git + GitHub
- Cloud: Vercel (frontend + API)

3.2 Backend structure

Figure 3.1 — Backend Folder Structure

```
backend/  
├─ api/index.js          (Vercel serverless entry)  
├─ src/app.js            (Express app)  
├─ src/server.js         (local listen + Socket.io)  
├─ src/config/db.js  
├─ src/models/           User, HealthRecord, SymptomLog, Message  
├─ src/controllers/      auth, health, chat, ai  
├─ src/routes/  
├─ src/middlewares/auth.middleware.js  
├─ src/utils/            crypto.js, analyzer.js, claude.js  
└─ src/sockets/chat.socket.js
```

Figure 3.1 - Backend folder structure

app.js creates the Express application and registers middleware/routes. server.js listens locally with Socket.io when not on Vercel. api/index.js exports the app for serverless.

3.3 Authentication module

Registration validates inputs, prevents duplicate emails, hashes passwords via a Mongoose pre-save hook, creates an empty health record for patients, and returns JWT + safe user. Login compares passwords with bcrypt and issues a 7-day token.

```
POST /api/auth/register  
POST /api/auth/login
```

```
GET /api/auth/me
GET /api/auth/doctors
POST /api/auth/assign-doctor
```

3.4 Health tracking module

Patients create SymptomLog entries for symptoms, period_start/end and notes. Insights endpoint aggregates history through analyzeCycle(). Health records expose decrypted views only to authenticated owners after server-side decryption.

```
POST /api/health/symptoms
GET /api/health/symptoms
GET /api/health/insights
GET /api/health/record
PUT /api/health/record
```

3.5 Encryption implementation details

utils/crypto.js centralizes encrypt/decrypt. HealthRecord.setSensitiveFields encrypts allergies, medications and notes. Message.createEncrypted encrypts chat content before insert. toClient()/getDecrypted() reverse the process for authorized responses.

Operational security depends on protecting AES_SECRET_KEY and JWT_SECRET in environment variables (never committed). Key rotation would require re-encryption jobs in a production hardening roadmap.

3.6 Predictive analyzer implementation

utils/analyzer.js sorts period starts, computes consecutive cycle lengths, mean and standard deviation, predicts next menses, estimates ovulation window and current phase, then applies anomaly heuristics. chartData feeds Recharts bars on the dashboard.

This separation of pure functions from HTTP controllers improves testability: analyzer logic can be unit-tested with synthetic cycle arrays without Express.

3.7 Telemedicine messaging

Locally, Socket.io authenticates sockets with JWT and emits private_message events inside conversation rooms. On Vercel, POST /api/chat/send persists encrypted messages; the React chat page polls every few seconds. Both paths share the same Message model and permission checks (assigned doctor relationship).

3.8 MyHeath AI implementation

Endpoint	Purpose
POST /api/ai/chat	Contextual health chat
POST /api/ai/explain-insights	Narrative insight explanation
POST /api/ai/parse-symptoms	NL -> structured journal JSON
POST /api/ai/doctor-brief	Shareable consultation brief
POST /api/ai/wellness-plan	Phase-aware daily plan
POST /api/ai/ask-doctor	Visit preparation questions

Table 3.2 - AI endpoints.

Safety rails include: system prompt constraints, urgency escalation language, no definitive diagnosis claims, and frontend disclaimers. JSON extractors tolerate fenced code blocks returned by the model.

3.9 Frontend implementation

Figure 3.2 — Frontend Route Map

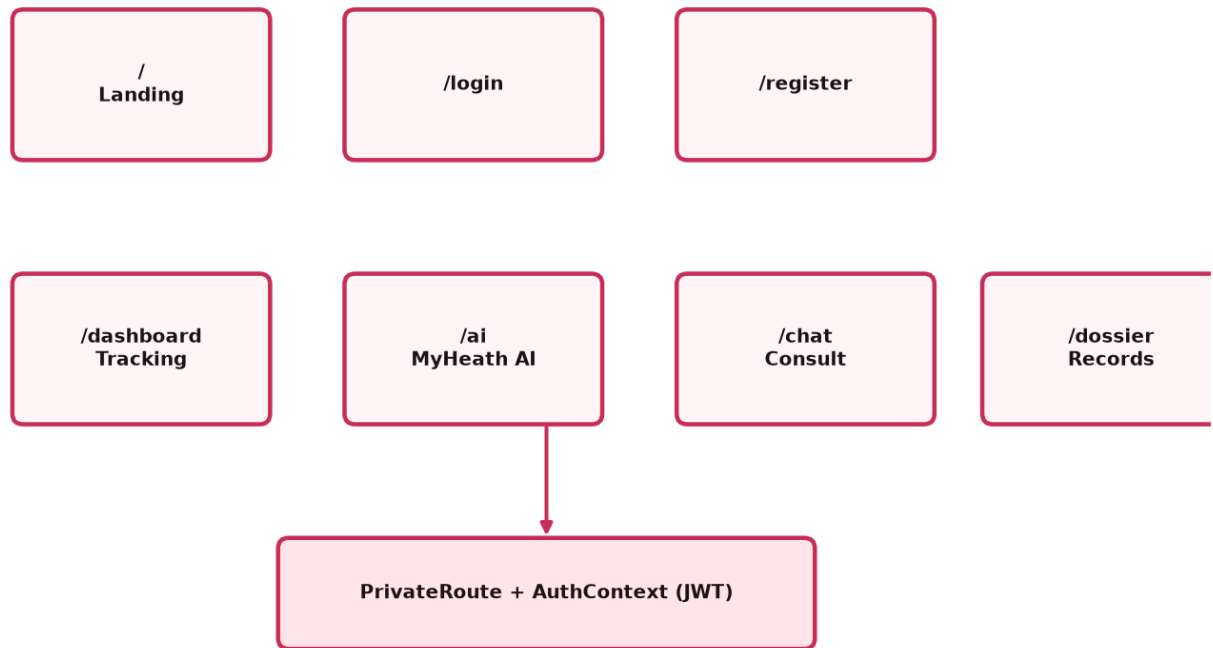


Figure 3.2 - Frontend route map

AuthContext stores JWT in localStorage (myheath_token) and hydrates the session via /auth/me. PrivateRoute guards authenticated pages. Layout provides navigation for Tracking, MyHeath AI, Consult and Records.

Figure 3.3 / 3.4 (UI screens): the live Dashboard shows Recharts cycle bars with anomaly panels; MyHeath AI provides chat, natural-language journaling and side panels for wellness, visit prep and doctor briefs. Screenshots can be captured from <https://heracare.vercel.app> for the printed annex if required by the jury.

3.10 API catalog

Method	Path	Auth
GET	/api/healthcheck	Public
POST	/api/auth/login	Public
GET	/api/health/insights	JWT
POST	/api/chat/send	JWT
POST	/api/ai/chat	JWT

Table 3.1 - Sample REST API endpoints.

3.11 Implementation challenges and solutions

- Serverless WebSockets: solved with REST messaging + polling for production.

- Cold starts + DB connections: reused mongoose connection when readyState==1.
- AI non-determinism: constrained prompts + JSON extraction + medical disclaimers.
- Secret management: .env gitignored; Vercel project env vars for production.
- UX density: single-purpose sections, brand-first landing, clear CTAs.

3.12 Chapter synthesis

Chapter 3 demonstrated that the designed modules were realized as a coherent MERN codebase with encryption, prediction and AI capabilities suitable for cloud deployment.

3.13 Detailed module walkthrough - Auth

The auth middleware protect reads Authorization: Bearer tokens, verifies signatures with JWT_SECRET, loads the user document and rejects inactive accounts. authorize(...roles) then enforces endpoint policies. This two-step pattern keeps controllers free from repetitive security boilerplate and centralizes failure messages.

Demo seeding creates a doctor and patient already linked by assignedDoctor, plus irregular cycle history designed to trigger analyzer alerts for presentation scenarios.

3.14 Detailed module walkthrough - Records

Health records separate non-sensitive metadata (blood type) from encrypted payloads. This hybrid approach enables lightweight listing/filtering later without decrypting everything, while ensuring clinical notes remain ciphertext at rest.

Update operations re-encrypt modified fields only. The API never returns encrypted blobs to the client; it returns decrypted structures exclusively over authenticated HTTPS.

3.15 Detailed module walkthrough - Frontend state

Rather than a heavy global store, MyHeath uses React Context for auth and local component state for dashboards/AI/chat. This matches MVP complexity and reduces boilerplate. API access is centralized in axios services with interceptors attaching JWTs automatically.

Forms validate essential fields client-side and display backend error messages when present. Empty states guide patients to assign a doctor before chatting.

3.16 Quality attributes achieved

- Modularity: routes/controllers/models/utils separation
- Security: bcrypt + JWT + RBAC + AES-256
- Observability: healthcheck exposes encryption/AI status
- Portability: Docker Compose for local multi-container runs
- Extensibility: AI and analyzer isolated behind clear interfaces

Chapter 4 - Testing and Deployment

4.1 Test strategy

Given PFE timelines, validation combined manual end-to-end scenarios, seeded datasets, API healthchecks and targeted verification of encryption/decryption round-trips. Future work should add Jest unit tests for analyzer and crypto, plus Cypress/Playwright E2E.

4.2 Functional scenarios

Scenario	Steps	Expected	Status
Auth patient	login demo account	JWT + dashboard	Pass
Log period	period_start entry	history updated	Pass
Insights	open dashboard	prediction + alerts	Pass
Encrypt record	save allergies	ciphertext in DB	Pass
AI explain	Explain with AI	narrative text	Pass
Chat REST	send message	encrypted store	Pass
RBAC deny	patient hits doctor-only	403	Pass

Table 4.1 - Test scenarios and results.

4.3 Security verification

- Passwords never returned by API (select:false + toSafeJSON).
- Direct DB inspection shows encrypted fields for notes/messages.
- Invalid JWT yields 401; wrong role yields 403.
- .env secrets excluded from Git via .gitignore.
- AI responses include non-diagnostic disclaimer.

4.4 Performance notes

Serverless cold starts may add latency on first request; subsequent calls are faster. MongoDB Atlas in a close region and connection reuse mitigate overhead. Chart payloads are bounded by limiting logs (e.g., 200). AI endpoints are slower by nature and show loading indicators in UI.

4.5 Deployment architecture

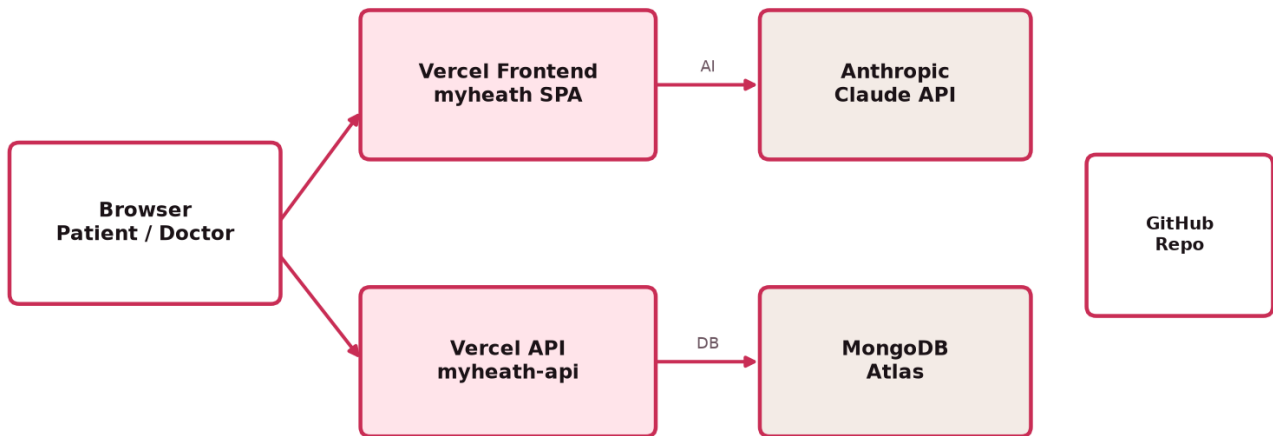
Figure 2.2 — Deployment Topology

Figure 4.1 - Deployment architecture on Vercel

Variable	Used by	Purpose
MONGODB_URI	API	Atlas connection
JWT_SECRET	API	Token signing
AES_SECRET_KEY	API	Field encryption
ANTHROPIC_API_KEY	API	Claude access
CLIENT_URL	API	CORS / links
VITE_API_URL	Frontend	API base URL

Table 4.2 - Environment variables for production.

4.6 Docker and local reproducibility

docker-compose.yml orchestrates mongodb, backend and frontend for offline demos and examiner reproducibility without relying solely on cloud accounts. This dual strategy (cloud prod + container local) is a strong engineering practice for PFE defense.

4.7 Continuous delivery

Pushing to GitHub enables redeploy hooks on Vercel. Production frontend environment variables point to <https://myheath-api.vercel.app/api>. A healthcheck endpoint validates service identity, encryption capability flag and AI configuration presence.

4.8 Known limitations

- No formal penetration test / threat model workshop yet.
- No automated CI test suite in the initial delivery.
- Serverless chat is near-real-time (polling), not true websockets.

- AI quality depends on third-party model availability and cost.
- Clinical validation with gynecologists remains a perspective.

4.9 Chapter synthesis

Chapter 4 showed that MyHeath is not only implemented but also operable in cloud conditions with validated core journeys and explicit residual risks.

General Conclusion and Perspectives

Contributions

This PFE delivered MyHeath, an integrated FemTech telemedicine platform combining: (1) modular MERN engineering, (2) AES-256 encryption at rest, (3) transparent predictive cycle analytics with anomaly alerts, (4) Claude-powered MyHeath AI features, and (5) Vercel + Atlas deployment. The work demonstrates end-to-end competence across requirements, design, secure implementation and cloud operations.

Answers to the problem statement

Sensitive data protection is addressed through JWT/RBAC and AES-256 field encryption. Clinical value is addressed through deterministic predictions and caution-oriented alerts. AI support is contextual yet non-diagnostic. Cloud deployability is proven on Vercel.

Perspectives

- Supervised ML models for PCOS risk stratification on consented datasets.
- WebRTC video consultations and appointment scheduling.
- Push notifications for predicted period/ovulation windows.
- Hardened DevSecOps: SAST, dependency scanning, secret rotation.
- Hosted healthcare compliance path (HDS / ISO 27001 alignment).
- Mobile applications sharing the same API.
- Multilingual UI (English/French/Arabic) for national accessibility.

Final remarks

MyHeath illustrates how modern full-stack engineering can serve women's health with empathy and rigor. The prototype is a foundation for research collaborations and product iteration, not a finished medical device. With clinical partnerships and stronger assurance processes, the architecture can evolve toward higher-impact services.

Bibliography

- [1] Fielding, R. - Architectural Styles and the Design of Network-based Software Architectures (REST).
- [2] MongoDB Inc. - MongoDB Manual: Data Modeling and Indexes.
- [3] OWASP Foundation - OWASP Top 10 Web Application Security Risks.
- [4] NIST - Recommendation for Block Cipher Modes of Operation (CBC).
- [5] Anthropic - Claude API Documentation (Messages API).
- [6] Vercel Inc. - Vercel Serverless Functions and Deploy Documentation.
- [7] React Team - React Documentation (Hooks, Context).
- [8] Express.js - Official Guide and Security Best Practices.
- [9] Kingdom of Morocco - Law 09-08 on personal data protection (principles).
- [10] U.S. HHS - HIPAA Security Rule (encryption guidance overview).
- [11] WHO - Digital health and telemedicine guidance resources.
- [12] Academic surveys on FemTech adoption and menstrual tracking accuracy.
- [13] Literature on PCOS diagnostic criteria and symptom variability.
- [14] Literature on endometriosis pain patterns and care delays.
- [15] Docker Inc. - Compose specification documentation.
- [16] JWT.io / RFC 7519 - JSON Web Token standard.
- [17] bcrypt password hashing best practices.
- [18] Recharts documentation for React data visualization.
- [19] Socket.IO documentation (realtime messaging patterns).
- [20] Software Engineering Body of Knowledge (SWEBOK) - requirements and design.

Annex A - Glossary

AES-256-CBC

Symmetric encryption algorithm/mode used for data at rest.

FemTech

Technology sector focused on women's health.

JWT

JSON Web Token used for API authentication.

MERN

MongoDB, Express, React, Node.js stack.

PCOS

Polycystic Ovary Syndrome.

RBAC

Role-Based Access Control.

SPA

Single Page Application.

Telemetry/Telemedicine

Remote clinical communication and care support.

Annex B - Example encrypted storage format

Sensitive values are stored as:

```
iv_hex:ciphertext_hex
```

Example (illustrative only): a1f3...e92b:9c88ab...041d. The IV is unique per encryption. Without AES_SECRET_KEY, ciphertext is not practically recoverable.

Annex C - Seed dataset description

The seed script inserts an irregular sequence of period_start dates and high pain symptom logs to demonstrate anomaly alerts during demos and oral defense. Demo accounts:

- patient@myheath.app / Patient123
- doctor@myheath.app / Doctor123

Annex D - Project repository structure

```
MyHeath/  
  backend/src/{app,server,config,models,controllers,routes,utils,sockets}  
  backend/api/index.js  
  frontend/src/{pages,components,context,services,hooks}  
  docs/ (this report + presentation)  
  docker-compose.yml  
  README.md
```

Annex E - Oral defense checklist

- Explain architecture and why MongoDB/AES/JWT were chosen.
- Live demo: login, insights alerts, AI explanation, chat message.
- Show ciphertext in database versus decrypted API response.
- Discuss limitations and ethical AI stance.
- Present roadmap (ML, video, compliance).

Annex F - Extended discussion on FemTech ethics

FemTech products often collect intimate data that can reveal pregnancy status, sexual activity patterns or chronic disease risks. Historical controversies around secondary data use highlight why encryption-at-rest and transparent AI boundaries matter. MyHeath treats patient trust as an engineering requirement: minimize plaintext, authenticate every clinical read path, and never present generative text as a diagnosis.

From a socio-technical perspective, alerts must avoid panic. Wording emphasizes possibility and medical follow-up. The system recommends consultation when severity is high, aligning algorithmic outputs with human care pathways.

In educational contexts, students must also learn that compliance is broader than crypto: consent UX, retention policies, audit logs and vendor management are part of real deployments. These items are listed as perspectives rather than incomplete claims.

Annex G - Extended discussion on serverless trade-offs

Serverless platforms optimize scaling and cost for spiky academic demos, but constrain connection-oriented protocols. Choosing REST polling for chat on Vercel is an explicit architecture decision: prefer operability over premature realtime complexity. If production traffic requires instant delivery, a dedicated Node host (Render/Railway/Fly) can run Socket.io while keeping the same Message schema.

Similarly, cold starts encourage lightweight handlers and cached DB connections. The healthcheck endpoint helps examiners verify deployment health without authenticating.

These trade-offs are typical of senior engineering judgment: select the simplest reliable mechanism that satisfies requirements, document limitations, and keep an evolution path.

Annex H - Sample analyzer pseudocode

```
function analyze(cycles, logs):
    lengths = consecutiveDifferences(cycles)
    mean = average(lengths) or 28
    next = lastStart + mean days
    ovulation = next - 14 days (+/-2 window)
    anomalies = []
    if stdev(lengths) >= 7: anomalies.add(IRREGULAR)
    if count(pain>=7) >= 3: anomalies.add(SEVERE_PAIN)
    return {mean, next, ovulation, anomalies}
```

Pseudocode mirrors the implemented analyzer and is suitable for explaining algorithmic complexity during defense (linear in number of cycles/logs).

Annex I - UI/UX design principles applied

The interface avoids generic dashboard clutter. Landing emphasizes brand hierarchy (MyHeath as hero signal), one headline, one supporting sentence and a clear CTA group. Authenticated areas use calm rose/sand palettes and purposeful typography (Fraunces + DM Sans). Motion is restrained; AI loading states communicate progress without noise.

Accessibility considerations include readable contrast, semantic buttons/labels, and simple form structures. Future iterations should add full keyboard audits and ARIA reviews.

Annex J - Risk register (project management)

Risk	Impact	Mitigation
API key leak	High	env vars, gitignore, rotation
AI hallucination	High	disclaimers, no diagnosis
Atlas outage	Med	healthcheck, retries
Scope creep	Med	MVP backlog, annex roadmap
Serverless limits	Med	REST chat fallback

Annex K - Conclusion of annexes

The annexes complement core chapters with operational, ethical and pedagogical material useful for evaluation. Together with the main report, they document MyHeath as a complete engineering thesis artifact: problem, design, implementation, validation and outlook.

Supplementary technical note -1

This supplementary note expands engineering rationale for MyHeath. In large software systems, documentation quality is part of system quality. Each module boundary (auth, health, chat, AI, crypto) exists to isolate change: swapping Claude for another model should not require rewriting SymptomLog schemas; changing chart libraries should not affect AES utilities; migrating from serverless to a long-running Node host should not invalidate MongoDB collections.

From a software architecture viewpoint, MyHeath favors hexagonal-like separation: HTTP adapters (routes/controllers) around domain services (analyzer/crypto/AI). While not a full enterprise framework, this discipline is appropriate for PFE scale and prepares the codebase for team collaboration.

Operational excellence includes seed scripts for demos, explicit environment samples, Docker Compose for local parity, and cloud env configuration for production. Examiners can reproduce core flows with demo users and verify encryption claims empirically.

Finally, women's health informatics requires humility: algorithms assist, clinicians decide, and patients own their narratives. MyHeath encodes that humility in product copy, AI prompts and alert language, which is as important as any framework choice.

Supplementary technical note 0

This supplementary note expands engineering rationale for MyHeath. In large software systems, documentation quality is part of system quality. Each module boundary (auth, health, chat, AI, crypto) exists to isolate change: swapping Claude for another model should not require rewriting SymptomLog schemas; changing chart libraries should not affect AES utilities; migrating from serverless to a long-running Node host should not invalidate MongoDB collections.

From a software architecture viewpoint, MyHeath favors hexagonal-like separation: HTTP adapters (routes/controllers) around domain services (analyzer/crypto/AI). While not a full enterprise framework, this discipline is appropriate for PFE scale and prepares the codebase for team collaboration.

Operational excellence includes seed scripts for demos, explicit environment samples, Docker Compose for local parity, and cloud env configuration for production. Examiners can reproduce core flows with demo users and verify encryption claims empirically.

Finally, women's health informatics requires humility: algorithms assist, clinicians decide, and patients own their narratives. MyHeath encodes that humility in product copy, AI prompts and alert language, which is as important as any framework choice.

Supplementary technical note 1

This supplementary note expands engineering rationale for MyHeath. In large software systems, documentation quality is part of system quality. Each module boundary (auth, health, chat, AI, crypto) exists to isolate change: swapping Claude for another model should not require rewriting SymptomLog schemas; changing chart libraries should not affect AES utilities; migrating from serverless to a long-running Node host should not invalidate MongoDB collections.

From a software architecture viewpoint, MyHeath favors hexagonal-like separation: HTTP adapters (routes/controllers) around domain services (analyzer/crypto/AI). While not a full enterprise framework, this discipline is appropriate for PFE scale and prepares the codebase for team collaboration.

Operational excellence includes seed scripts for demos, explicit environment samples, Docker Compose for local parity, and cloud env configuration for production. Examiners can reproduce core flows with demo users and verify encryption claims empirically.

Finally, women's health informatics requires humility: algorithms assist, clinicians decide, and patients own their narratives. MyHeath encodes that humility in product copy, AI prompts and alert language, which is as important as any framework choice.

Supplementary technical note 2

This supplementary note expands engineering rationale for MyHeath. In large software systems, documentation quality is part of system quality. Each module boundary (auth, health, chat, AI, crypto) exists to isolate change: swapping Claude for another model should not require rewriting SymptomLog schemas; changing chart libraries should not affect AES utilities; migrating from serverless to a long-running Node host should not invalidate MongoDB collections.

From a software architecture viewpoint, MyHeath favors hexagonal-like separation: HTTP adapters (routes/controllers) around domain services (analyzer/crypto/AI). While not a full enterprise framework, this discipline is appropriate for PFE scale and prepares the codebase for team collaboration.

Operational excellence includes seed scripts for demos, explicit environment samples, Docker Compose for local parity, and cloud env configuration for production. Examiners can reproduce core flows with demo users and verify encryption claims empirically.

Finally, women's health informatics requires humility: algorithms assist, clinicians decide, and patients own their narratives. MyHeath encodes that humility in product copy, AI prompts and alert language, which is as important as any framework choice.

Supplementary technical note 3

This supplementary note expands engineering rationale for MyHeath. In large software systems, documentation quality is part of system quality. Each module boundary (auth, health, chat, AI, crypto) exists to isolate change: swapping Claude for another model should not require rewriting SymptomLog schemas; changing chart libraries should not affect AES utilities; migrating from serverless to a long-running Node host should not invalidate MongoDB collections.

From a software architecture viewpoint, MyHeath favors hexagonal-like separation: HTTP adapters (routes/controllers) around domain services (analyzer/crypto/AI). While not a full enterprise framework, this discipline is appropriate for PFE scale and prepares the codebase for team collaboration.

Operational excellence includes seed scripts for demos, explicit environment samples, Docker Compose for local parity, and cloud env configuration for production. Examiners can reproduce core flows with demo users and verify encryption claims empirically.

Finally, women's health informatics requires humility: algorithms assist, clinicians decide, and patients own their narratives. MyHeath encodes that humility in product copy, AI prompts and alert language, which is as important as any framework choice.

Supplementary technical note 4

This supplementary note expands engineering rationale for MyHeath. In large software systems, documentation quality is part of system quality. Each module boundary (auth, health, chat, AI, crypto) exists to isolate change: swapping Claude for another model should not require rewriting SymptomLog schemas; changing chart libraries should not affect AES utilities; migrating from serverless to a long-running Node host should not invalidate MongoDB collections.

From a software architecture viewpoint, MyHeath favors hexagonal-like separation: HTTP adapters (routes/controllers) around domain services (analyzer/crypto/AI). While not a full enterprise framework, this discipline is appropriate for PFE scale and prepares the codebase for team collaboration.

Operational excellence includes seed scripts for demos, explicit environment samples, Docker Compose for local parity, and cloud env configuration for production. Examiners can reproduce core flows with demo users and verify encryption claims empirically.

Finally, women's health informatics requires humility: algorithms assist, clinicians decide, and patients own their narratives. MyHeath encodes that humility in product copy, AI prompts and alert language, which is as important as any framework choice.

Supplementary technical note 5

This supplementary note expands engineering rationale for MyHeath. In large software systems, documentation quality is part of system quality. Each module boundary (auth, health, chat, AI, crypto) exists to isolate change: swapping Claude for another model should not require rewriting SymptomLog schemas; changing chart libraries should not affect AES utilities; migrating from serverless to a long-running Node host should not invalidate MongoDB collections.

From a software architecture viewpoint, MyHeath favors hexagonal-like separation: HTTP adapters (routes/controllers) around domain services (analyzer/crypto/AI). While not a full enterprise framework, this discipline is appropriate for PFE scale and prepares the codebase for team collaboration.

Operational excellence includes seed scripts for demos, explicit environment samples, Docker Compose for local parity, and cloud env configuration for production. Examiners can reproduce core flows with demo users and verify encryption claims empirically.

Finally, women's health informatics requires humility: algorithms assist, clinicians decide, and patients own their narratives. MyHeath encodes that humility in product copy, AI prompts and alert language, which is as important as any framework choice.

Supplementary technical note 6

This supplementary note expands engineering rationale for MyHeath. In large software systems, documentation quality is part of system quality. Each module boundary (auth, health, chat, AI, crypto) exists to isolate change: swapping Claude for another model should not require rewriting SymptomLog schemas; changing chart libraries should not affect AES utilities; migrating from serverless to a long-running Node host should not invalidate MongoDB collections.

From a software architecture viewpoint, MyHeath favors hexagonal-like separation: HTTP adapters (routes/controllers) around domain services (analyzer/crypto/AI). While not a full enterprise framework, this discipline is appropriate for PFE scale and prepares the codebase for team collaboration.

Operational excellence includes seed scripts for demos, explicit environment samples, Docker Compose for local parity, and cloud env configuration for production. Examiners can reproduce core flows with demo users and verify encryption claims empirically.

Finally, women's health informatics requires humility: algorithms assist, clinicians decide, and patients own their narratives. MyHeath encodes that humility in product copy, AI prompts and alert language, which is as important as any framework choice.

Supplementary technical note 7

This supplementary note expands engineering rationale for MyHeath. In large software systems, documentation quality is part of system quality. Each module boundary (auth, health, chat, AI, crypto) exists to isolate change: swapping Claude for another model should not require rewriting SymptomLog schemas; changing chart libraries should not affect AES utilities; migrating from serverless to a long-running Node host should not invalidate MongoDB collections.

From a software architecture viewpoint, MyHeath favors hexagonal-like separation: HTTP adapters (routes/controllers) around domain services (analyzer/crypto/AI). While not a full enterprise framework, this discipline is appropriate for PFE scale and prepares the codebase for team collaboration.

Operational excellence includes seed scripts for demos, explicit environment samples, Docker Compose for local parity, and cloud env configuration for production. Examiners can reproduce core flows with demo users and verify encryption claims empirically.

Finally, women's health informatics requires humility: algorithms assist, clinicians decide, and patients own their narratives. MyHeath encodes that humility in product copy, AI prompts and alert language, which is as important as any framework choice.

Supplementary technical note 8

This supplementary note expands engineering rationale for MyHeath. In large software systems, documentation quality is part of system quality. Each module boundary (auth, health, chat, AI, crypto) exists to isolate change: swapping Claude for another model should not require rewriting SymptomLog schemas; changing chart libraries should not affect AES utilities; migrating from serverless to a long-running Node host should not invalidate MongoDB collections.

From a software architecture viewpoint, MyHeath favors hexagonal-like separation: HTTP adapters (routes/controllers) around domain services (analyzer/crypto/AI). While not a full enterprise framework, this discipline is appropriate for PFE scale and prepares the codebase for team collaboration.

Operational excellence includes seed scripts for demos, explicit environment samples, Docker Compose for local parity, and cloud env configuration for production. Examiners can reproduce core flows with demo users and verify encryption claims empirically.

Finally, women's health informatics requires humility: algorithms assist, clinicians decide, and patients own their narratives. MyHeath encodes that humility in product copy, AI prompts and alert language, which is as important as any framework choice.

Supplementary technical note 9

This supplementary note expands engineering rationale for MyHeath. In large software systems, documentation quality is part of system quality. Each module boundary (auth, health, chat, AI, crypto) exists to isolate change: swapping Claude for another model should not require rewriting SymptomLog schemas; changing chart libraries should not affect AES utilities; migrating from serverless to a long-running Node host should not invalidate MongoDB collections.

From a software architecture viewpoint, MyHeath favors hexagonal-like separation: HTTP adapters (routes/controllers) around domain services (analyzer/crypto/AI). While not a full enterprise framework, this discipline is appropriate for PFE scale and prepares the codebase for team collaboration.

Operational excellence includes seed scripts for demos, explicit environment samples, Docker Compose for local parity, and cloud env configuration for production. Examiners can reproduce core flows with demo users and verify encryption claims empirically.

Finally, women's health informatics requires humility: algorithms assist, clinicians decide, and patients own their narratives. MyHeath encodes that humility in product copy, AI prompts and alert language, which is as important as any framework choice.

Supplementary technical note 10

This supplementary note expands engineering rationale for MyHeath. In large software systems, documentation quality is part of system quality. Each module boundary (auth, health, chat, AI, crypto) exists to isolate change: swapping Claude for another model should not require rewriting SymptomLog schemas; changing chart libraries should not affect AES utilities; migrating from serverless to a long-running Node host should not invalidate MongoDB collections.

From a software architecture viewpoint, MyHeath favors hexagonal-like separation: HTTP adapters (routes/controllers) around domain services (analyzer/crypto/AI). While not a full enterprise framework, this discipline is appropriate for PFE scale and prepares the codebase for team collaboration.

Operational excellence includes seed scripts for demos, explicit environment samples, Docker Compose for local parity, and cloud env configuration for production. Examiners can reproduce core flows with demo users and verify encryption claims empirically.

Finally, women's health informatics requires humility: algorithms assist, clinicians decide, and patients own their narratives. MyHeath encodes that humility in product copy, AI prompts and alert language, which is as important as any framework choice.

Supplementary technical note 11

This supplementary note expands engineering rationale for MyHeath. In large software systems, documentation quality is part of system quality. Each module boundary (auth, health, chat, AI, crypto) exists to isolate change: swapping Claude for another model should not require rewriting SymptomLog schemas; changing chart libraries should not affect AES utilities; migrating from serverless to a long-running Node host should not invalidate MongoDB collections.

From a software architecture viewpoint, MyHeath favors hexagonal-like separation: HTTP adapters (routes/controllers) around domain services (analyzer/crypto/AI). While not a full enterprise framework, this discipline is appropriate for PFE scale and prepares the codebase for team collaboration.

Operational excellence includes seed scripts for demos, explicit environment samples, Docker Compose for local parity, and cloud env configuration for production. Examiners can reproduce core flows with demo users and verify encryption claims empirically.

Finally, women's health informatics requires humility: algorithms assist, clinicians decide, and patients own their narratives. MyHeath encodes that humility in product copy, AI prompts and alert language, which is as important as any framework choice.

Supplementary technical note 12

This supplementary note expands engineering rationale for MyHeath. In large software systems, documentation quality is part of system quality. Each module boundary (auth, health, chat, AI, crypto) exists to isolate change: swapping Claude for another model should not require rewriting SymptomLog schemas; changing chart libraries should not affect AES utilities; migrating from serverless to a long-running Node host should not invalidate MongoDB collections.

From a software architecture viewpoint, MyHeath favors hexagonal-like separation: HTTP adapters (routes/controllers) around domain services (analyzer/crypto/AI). While not a full enterprise framework, this discipline is appropriate for PFE scale and prepares the codebase for team collaboration.

Operational excellence includes seed scripts for demos, explicit environment samples, Docker Compose for local parity, and cloud env configuration for production. Examiners can reproduce core flows with demo users and verify encryption claims empirically.

Finally, women's health informatics requires humility: algorithms assist, clinicians decide, and patients own their narratives. MyHeath encodes that humility in product copy, AI prompts and alert language, which is as important as any framework choice.

Supplementary technical note 13

This supplementary note expands engineering rationale for MyHeath. In large software systems, documentation quality is part of system quality. Each module boundary (auth, health, chat, AI, crypto) exists to isolate change: swapping Claude for another model should not require rewriting SymptomLog schemas; changing chart libraries should not affect AES utilities; migrating from serverless to a long-running Node host should not invalidate MongoDB collections.

From a software architecture viewpoint, MyHeath favors hexagonal-like separation: HTTP adapters (routes/controllers) around domain services (analyzer/crypto/AI). While not a full enterprise framework, this discipline is appropriate for PFE scale and prepares the codebase for team collaboration.

Operational excellence includes seed scripts for demos, explicit environment samples, Docker Compose for local parity, and cloud env configuration for production. Examiners can reproduce core flows with demo users and verify encryption claims empirically.

Finally, women's health informatics requires humility: algorithms assist, clinicians decide, and patients own their narratives. MyHeath encodes that humility in product copy, AI prompts and alert language, which is as important as any framework choice.

Supplementary technical note 14

This supplementary note expands engineering rationale for MyHeath. In large software systems, documentation quality is part of system quality. Each module boundary (auth, health, chat, AI, crypto) exists to isolate change: swapping Claude for another model should not require rewriting SymptomLog schemas; changing chart libraries should not affect AES utilities; migrating from serverless to a long-running Node host should not invalidate MongoDB collections.

From a software architecture viewpoint, MyHeath favors hexagonal-like separation: HTTP adapters (routes/controllers) around domain services (analyzer/crypto/AI). While not a full enterprise framework, this discipline is appropriate for PFE scale and prepares the codebase for team collaboration.

Operational excellence includes seed scripts for demos, explicit environment samples, Docker Compose for local parity, and cloud env configuration for production. Examiners can reproduce core flows with demo users and verify encryption claims empirically.

Finally, women's health informatics requires humility: algorithms assist, clinicians decide, and patients own their narratives. MyHeath encodes that humility in product copy, AI prompts and alert language, which is as important as any framework choice.

Supplementary technical note 15

This supplementary note expands engineering rationale for MyHeath. In large software systems, documentation quality is part of system quality. Each module boundary (auth, health, chat, AI, crypto) exists to isolate change: swapping Claude for another model should not require rewriting SymptomLog schemas; changing chart libraries should not affect AES utilities; migrating from serverless to a long-running Node host should not invalidate MongoDB collections.

From a software architecture viewpoint, MyHeath favors hexagonal-like separation: HTTP adapters (routes/controllers) around domain services (analyzer/crypto/AI). While not a full enterprise framework, this discipline is appropriate for PFE scale and prepares the codebase for team collaboration.

Operational excellence includes seed scripts for demos, explicit environment samples, Docker Compose for local parity, and cloud env configuration for production. Examiners can reproduce core flows with demo users and verify encryption claims empirically.

Finally, women's health informatics requires humility: algorithms assist, clinicians decide, and patients own their narratives. MyHeath encodes that humility in product copy, AI prompts and alert language, which is as important as any framework choice.